Beyond Memory Pressure: Latent-State Gating as a Hardware-Native Trigger for Corrective Retrieval Across the 2026 Attention-Design Spectrum

Author One
Affiliation
author1@example.org

Working draft, May 13, 2026

Abstract

Corrective retrieval-augmented generation (**??**) has emerged as the dominant pattern for keeping small and medium-sized language models factual under long agentic reasoning, but every existing trigger family — discourse markers (ReaLM-Retrieve), control tokens (GRIP) (**??**) — operates over generated text and therefore cannot fire until the model has already committed at least one token of a hallucinated span. We study a family of triggers that observes the model's hidden state directly at logical step boundaries, i.e. the Concept Representation Vector (CRV). The CRV is computed inside the forward pass and is therefore *hardware-native*: it is available strictly before any post-decode text-parsing baseline can fire. The contribution is three-fold.

First, we make explicit that *memory pressure is being solved by architecture, not the gate*. We measure 5 000-token KV-pressure curves across three points on the 2026 attention-design spectrum — a dense full-attention SLM (Phi-4-mini), a sliding-window-with-sparse-global MoE (Gemma 4 26B A4B), and a 3:1 Gated DeltaNet hybrid MoE (Qwen3.6-35B-A3B) — and show that the architectural design space already brackets memory pressure into three distinct regimes (128, 221, 27 KB/token observed). The story decomposes cleanly into two independent optimization axes: *compute boundedness* and *memory boundedness*, with sliding-window attention bounding only the first in the standard reference implementation.

Second, we ship an architecture-agnostic adapter that captures CRV features through the public `output_hidden_states=True` API rather than `register_forward_hook`, with under 3% throughput overhead at 5 000-token context. The adapter, tracker, and trigger are five modules totalling ∼600 lines and run unchanged across all three architectural classes.

Third, we show empirically that *the L1 landmine fires, and is mitigated*. The naive last-layer mean-pooled CRV signal saturates on Qwen3.6's 3:1 Gated DeltaNet stack (mean cosine drift 0.479, near the upper end of the range across 1 213 committed steps), partially saturates on Gemma 4 (0.175), and remains clean on Phi-4-mini (0.024). Re-reading the CRV from the mid-stack ($\lfloor L/2 \rfloor$) instead of the last layer recovers a working signal: drift drops to 0.235 on Qwen3.6 (−51%), 0.106 on Gemma 4 (−39%), and stays at 0.033 on Phi-4-mini. Per-architecture noise floors converge to within an order of magnitude across the three classes after the mitigation, supporting the "architecture-agnostic, with the right layer subset" claim.

The paper's evaluation arm — Phase 4 in the engineering record — is deliberately preregistered here and not yet executed; we describe the HotpotQA / MuSiQue showdown design and the trigger calibration protocol as a roadmap, not as evidence.

1 Introduction

Retrieval-augmented generation has matured from one-shot retrieve-then-read (**?**) into agentic loops in which the model's own output stream determines, step by step, when to consult an external store and when to revise an in-progress reasoning trace (**????**). As the bulk of inference effort moves from the retrieve step to the generation trace, the engineering question has narrowed: *when, exactly, should a corrective retrieval (or any other heavy intervention) fire?*

Most existing trigger families read the question as a textual one. ReaLM (**?**) fires on either a small verifier's per-step uncertainty score or on discourse markers ("actually", "let me verify", "I need to check") in the generated text. GRIP-style control-token methods (**?**) learn to emit a special `<retrieve>` token whose appearance triggers the retrieval branch. Both are *post-decode* signals: by the time a discourse marker is present, or a control token is sampled, the model has already begun to commit to whatever path the trigger was supposed to interrupt.

The architectural pivot of 2026. Until recently, the headline cost of long agentic generation was the KV cache. The cache grows linearly with context length in standard softmax attention; on consumer-VRAM hardware the cache rather than the model weights becomes the binding constraint past a few thousand tokens. Two architectural lines have converged in 2025–2026 to attack this problem directly: *sliding-window plus sparse global attention* (Gemma 2/3/4 family (**??**)) and *hybrid linear attention* that interleaves a recurrent state-update layer (Gated DeltaNet (**??**)) with periodic full-attention layers, typically at a 3:1 ratio (**??**).

The relevance to RAG triggers is that *the cost the gate is supposed to save is, increasingly, not the cost we thought it was.* A gate that prevents a 5 000-token tail from accumulating KV is solving a problem that a Qwen3.6-grade architecture has already solved by construction (we measure 27 KB/token of KV growth at 5,000 tokens of decode, see §**??**). Conversely, on a sliding-window architecture in the standard `DynamicCache` reference implementation the local-layer cache does *not* actually evict, so the slope is the steepest of the three (§**??**). Per-token compute and memory are no longer the same problem: an architecture can bound one without bounding the other.

Cognitive efficiency, not memory pressure. We argue the relevant binding cost in the agentic-reasoning regime of 2026 is not memory but *cognitive efficiency* — the wasted thinking, the unnecessary corrective-retrieval round trips, the trajectory drift past the moment the model first becomes uncertain. None of these are alleviated by sliding-window attention or by a 3:1 hybrid stack. They are the residual cost after the architectural optimization, and they are paid in tokens, in latency, and (for deployed agents) in dollars.

Contribution. This paper contributes (a) a *bracketing analysis* of the 2026 attention-design space against the cost a RAG gate could plausibly save, showing the optimization is now two-dimensional; (b) an *architecture-agnostic CRV adapter* that captures hidden state at logical step boundaries through HF's `output_hidden_states=True` public API rather than `register_forward_hook`, working unchanged across dense, sliding-window, and linear-attention architectures with $\leq 3\%$ throughput regression at 5 000-token context; and (c) an empirical confirmation of the L1 landmine, paired with a working pre-routing mid-stack mitigation: mean-pool of the *last* layer's hidden state is dominated by MoE expert-routing variance on the linear-attention hybrid class (mean drift 0.479, near saturation), and reading from $\lfloor L/2 \rfloor$ recovers a working signal (mean drift 0.235), without per-architecture code changes. Both runs are kept in the paper because the contrast itself is the deliverable.

What this paper deliberately does *not* do. The empirical evaluation of the trigger family — earliness against ReaLM and GRIP simulations on a HotpotQA / MuSiQue split, trajectory drift conditioned on fire-time, and the per-architecture cost Pareto — is preregistered in §?? and is not yet executed. This draft positions the diagnostic infrastructure (§??, §??) and the design-space measurements (§??) as the empirical core, and frames the showdown arm as a roadmap. We commit to running it before submission.

2 Related Work

Adaptive RAG with reflection / control tokens. Self-RAG (?) introduces the now-standard pattern of training the generator to emit specialized reflection tokens (`<retrieve>`, `<critique>`, `<utility>`) that gate retrieval calls and segment-level beam search. CRAG (?) adds a lightweight 0.77 B-parameter retrieval evaluator that scores the relevance of each retrieved passage and dispatches one of three actions (CORRECT / INCORRECT / AMBIGUOUS) on threshold crossings, together with a decompose-then-recompose document filter. Both approaches operate on *textual* signals — the generator's emitted tokens or the retriever's output — and therefore fire after the relevant decode step rather than during it. Our contribution is orthogonal: latent-state gating fires on hidden states *before* the next token is sampled, and is therefore at least one decode step earlier than any text-parsing trigger.

Linear-attention hybrids. The 2025–2026 linear-attention revival is best documented in the Qwen3-Next family (?) and Kimi Linear (?), both of which adopt a 3:1 ratio of linear-attention to full-attention layers. The linear layers in both are descendants of Gated DeltaNet (?), itself an extension of Mamba2 (?) with a delta-rule update on a fixed-size recurrent state matrix. Sebastian Raschka's pedagogical treatment of the family (?) is the cleanest public reference for the 3:1 ratio claim and we use it as our source of truth for the architectural ratio in our Qwen3.6 measurements (§??). Sliding-window attention with a sparse global layer pattern is the Gemma 2/3/4 lineage default (???); we refer to it as "Standard MoE" in this paper because the attention pattern is implemented over a standard softmax attention path, with sliding only gating the visible KV window during the attention computation.

Hidden-state probes. Probing the internal states of language models for signals of truthfulness, factuality, and confidence has a long history (???). The closest prior to the per-step CRV adapter we ship is the literature on activation clustering at sentence boundaries; we borrow the high-level intuition that hidden states near generation boundaries are more information-rich than mid-token activations. We do not, however, require a separately trained probe — the trigger is an unsupervised threshold on a scalar derived from the CRV.

3 Background: attention geometry and KV growth

We summarize the mechanics necessary for the bracketing argument in §??.

Standard attention KV cache. For a transformer with L layers, h_{kv} key/value heads per layer, head dimension d_h, and bytes-per-element b (i.e. $b = 2$ for `bfloat16`), the per-token KV memory contribution of a forward pass that writes to the cache is

$$\text{KV}_{\text{full}}(L, h_{\text{kv}}, d_h, b) = 2 \cdot L \cdot h_{\text{kv}} \cdot d_h \cdot b \quad \text{bytes / token.} \tag{1}$$

The factor of 2 accounts for separate K and V tensors. After T generated tokens, total KV growth is T times (**??**), i.e. strictly linear in context length.

Sliding-window attention. A sliding window of size W bounds the *attention computation* at each layer to look at most W tokens back. In a faithful production implementation (e.g. vLLM (**?**), SGLang (**?**)) the cache is also evicted past W tokens, so the layer's KV contribution caps at $W \cdot 2 \cdot h_{\text{kv}} \cdot d_h \cdot b$ bytes. In the HuggingFace `transformers` `DynamicCache` reference path, which we use, no eviction occurs: the cache stores everything and the attention mask masks past positions out. We measure this directly in §**??**; it is a finding of independent interest.

Gated DeltaNet recurrence. A Gated DeltaNet layer (**??**) maintains a recurrent state matrix $S_t \in \mathbb{R}^{h \times d_h \times d_h}$ (per head, per channel) updated per token via a delta rule:

$$\delta_t = (v_t - S_{t-1}\, k_t)\, \beta_t, \tag{2}$$

$$S_t = \alpha_t \cdot S_{t-1} + k_t\, \delta_t^{\top}, \tag{3}$$

where α_t is a per-head scalar decay, β_t is a per-channel update gate, and $k_t, v_t \in \mathbb{R}^{d_h}$ are the per-token key and value (after a small convolution mixer (**?**)). The per-layer memory cost is therefore $\mathcal{O}(h \cdot d_h^2)$, independent of context length; only the periodic full-attention layers contribute KV growth.

Mixture-of-experts (MoE). A standard MoE feed-forward block routes each token to a small subset of experts (*top-k routing*). Modern open-weight MoEs use a *grouped-expert* layout where each layer's expert weights are packed into a single $\mathbb{R}^{E \times d_{\text{ff}} \times d}$ parameter rather than E separate `nn.Linear` modules (**?**). This layout is incidental to the gating story but matters substantively for quantization tooling (§**??**), and we mention it because it constrained our hardware choice early in the project.

4 Method: latent-state gating and the CRV adapter

This section defines the Concept Representation Vector and the trigger family that consumes it. The implementation is engine- and architecture-agnostic by construction; §**??** describes the implementation; §**??** reports the diagnostic measurements.

4.1 The Concept Representation Vector

Let $\mathbf{h}_t^{(\ell)} \in \mathbb{R}^d$ denote the hidden state at layer ℓ for token t, where $\ell \in \{0, \ldots, L\}$ indexes into the HuggingFace `output.hidden_states` tuple ($\ell = 0$ is the embedding output and $\ell = L$ is the post-final-layer state). Let $\mathcal{S}_i = (t_{i-1} + 1, \ldots, t_i)$ denote the ith *logical step*, i.e. the contiguous span of tokens between two boundary tokens t_{i-1} and t_i (we define logical steps formally in §**??**).

For a configurable layer subset $\mathcal{L} \subseteq \{0, \ldots, L\}$ the per-step CRV is the concatenation of mean-pooled per-layer representations,

$$\mathbf{c}_i \;=\; \bigoplus_{\ell \in \mathcal{L}} \frac{1}{|\mathcal{S}_i|} \sum_{t \in \mathcal{S}_i} \mathbf{h}_t^{(\ell)}, \tag{4}$$

yielding $\mathbf{c}_i \in \mathbb{R}^{|\mathcal{L}|\cdot d}$. We report two layer-mode defaults: `last_layer` ($\mathcal{L} = \{L\}$) — the simplest baseline — and `mid_stack` ($\mathcal{L} = \{\lfloor L/2 \rfloor\}$), motivated by the L1 mitigation hypothesis that pre-routing mid-stack representations are less polluted by per-token expert-routing variability than the post-routing final layer.

4.2 Per-step features: drift and rolling variance

We derive two scalar features from the CRV stream.

Cosine drift between consecutive steps,

$$\text{drift}_{\cos i} \;=\; 1 - \frac{\mathbf{c}_{i-1}^{\top}\mathbf{c}_i}{\|\mathbf{c}_{i-1}\|_2 \cdot \|\mathbf{c}_i\|_2}, \quad \text{drift}_{\cos 1} := 0, \tag{5}$$

which is scale-invariant and bounded in $[0, 2]$; lower values mean the latent state moved less between adjacent steps.

Rolling L2-norm variance over the last W steps:

$$\text{Var}_i^{(W)} \;=\; \frac{1}{W' - 1} \sum_{j=i-W'+1}^{i} \Big(\|\mathbf{c}_j\|_2 \;-\; \overline{n}_i^{(W')} \Big)^2, \;\; W' = \min(W, i), \tag{6}$$

with $\overline{n}_i^{(W')}$ the rolling mean of $\|\mathbf{c}_\cdot\|_2$ over the same window. Larger W smooths; smaller W reacts faster. We use $W = 10$ throughout.

4.3 Step-boundary detector

The boundary between logical steps is the principal hyperparameter of this method. We support three operational definitions:

`sentence` fire on any token id whose decoded form (as a single token, with or without leading whitespace) is in the set { . ! ? \n \n\n } extended to common Unicode variants. Marker token ids are precomputed once at adapter init from the tokenizer and looked up in $\mathcal{O}(1)$ per token.

`think_block` fire on \n\n, `</think>`, `<think>`, and Markdown section delimiters (\n\#, \n\ * *). This is the natural unit for chain-of-thought-style reasoning models.

`fixed` fire every N tokens regardless of content. A calibration baseline; the gating signal that survives this strategy is free of any semantic step structure.

We report all results in this draft under `boundary=sentence`, which is the most natural fit for prose-heavy generation; sweeps across the three strategies are deferred to ablation (§**??**).

4.4 Trigger rule

We deliberately keep the trigger function deterministic and stateless. For thresholds τ_d, τ_v and a combination mode $m \in \{\text{OR}, \text{AND}, \text{DRIFT}, \text{VAR}, \text{NONE}\}$, the trigger fires at step i iff

$$\text{fired}_i \;=\; \begin{cases} \text{drift}_{\cos i} > \tau_d \;\vee\; \text{Var}_i^{(W)} > \tau_v & m = \text{OR}, \\ \text{drift}_{\cos i} > \tau_d \;\wedge\; \text{Var}_i^{(W)} > \tau_v & m = \text{AND}, \\ \text{drift}_{\cos i} > \tau_d & m = \text{DRIFT}, \\ \text{Var}_i^{(W)} > \tau_v & m = \text{VAR}, \\ \text{FALSE} & m = \text{NONE}. \end{cases} \tag{7}$$

Algorithm 1 Per-step CRV capture, manual decode loop.

Require: generator θ, tokenizer, prompt, $T_{\max}$, layer subset $\mathcal{L}$, boundary detector B, tracker $\mathcal{T}$ with rolling window W, trigger $\mathcal{F}$

Encode chat-templated prompt; let $\mathbf{x}$ be its token ids
cache $\leftarrow$ `None`; $i \leftarrow 0$
for $t = 1$ to $T_{\max}$ **do**
  $(\mathbf{logits}, \mathbf{H}, \text{cache}) \leftarrow \theta(\mathbf{x}, \text{cache}, \texttt{output_hidden_states} = \texttt{true})$
  $\mathbf{x} \leftarrow \arg\max \mathbf{logits}_{:,-1,:}$ ▷ greedy decode
  Capture $\{\mathbf{H}^{(\ell)}_{:,-1,:}\}_{\ell \in \mathcal{L}}$ to CPU fp32; append to $\mathcal{T}$'s buffer
  if $B(\mathbf{x})$ is a boundary **then**
    $i \leftarrow i + 1$
    $\mathbf{c}_i \leftarrow$ mean-pool of $\mathcal{T}$'s buffer per Eq. (**??**)
    Compute $\text{drift}_{\cos i}$ (**??**), $\text{Var}_i^{(W)}$ (**??**), fired_i (**??**)
    Drop $\mathcal{T}$'s buffer; emit per-step row to JSONL
  end if
end for

Threshold calibration is deferred to the empirical showdown (§??); reporting the firing rate at any particular threshold here would be premature, and we provide raw $\text{drift}_{\cos}$ and Var traces (Figures **??** and **??**) so the trigger threshold is a free parameter the reader can replace.

4.5 Adapter design and the implementation surface

We deliberately do *not* use `register_forward_hook` to capture hidden states. The HuggingFace public API exposes per-layer hidden states via `output.hidden_states` when `output_hidden_states=True` is passed to `model.forward`. That output tuple has the same length and same per-layer shape across all three architectural classes we study: dense full-attention, sliding-window-with-sparse-global, and Gated-DeltaNet-with-full hybrid. Using the public API costs us nothing (the captured representations are the same hidden states a forward hook would intercept) and gains us full architectural neutrality — the adapter contains zero per-architecture code, and we have verified end-to-end runs on all three classes (§**??**).

The cost we pay is that the model materializes *every* layer's hidden states before we sub-select; we cannot avoid the work even when $\mathcal{L} = \{L\}$. In §**??** we measure this overhead at $\sim 3\%$ of throughput at 5 000-token decode.

5 Experimental setup

5.1 Hardware and software stack

All measurements in this paper are produced on a single NVIDIA A800 PCIe 80 GB GPU (driver 550.90.07, CUDA runtime 12.4 reported by `torch`, 220 GiB host RAM, 32 cores) on a vast.ai-provisioned Linux container. The full 80 GB of VRAM is visible to the process (no cgroup limit). Software: Python 3.11.15, `torch 2.6.0+cu124`, `transformers 5.8.0`, `accelerate 1.13.0`, `bitsandbytes 0.49.2`, full lockfile in the artifacts.

We deliberately use a single high-VRAM card rather than a consumer-tier target; the bracketing argument (§**??**) requires loading three models sequentially with weight footprints of 7, 52, and 69 GB respectively in `bfloat16` native, and the diagnostic work in §**??** benefits from headroom.

Table 1: **Generator pool (three architectural classes spanning the 2026 attention-design spectrum).** All values read from the model's `config.json` (Qwen, Phi) or `AutoConfig.from_pretrained` (Gemma 4). MoE column gives *(experts active per token / total experts / shared experts)*. For Qwen3.6 we list global-attention layer indices verbatim from the model's `layer_types` array. For Gemma 4 the `num_global_key_value_heads` and `global_head_dim` in the global-layer rows differ from the sliding-layer values (8 KV heads, head_dim 256); only one is shown per architecture for brevity.

Model (HF id)	Class	Layers	Heads	KV	Hidden	MoE (act/tot/shr)	bf16 GB
`microsoft/Phi-4-mini-instruct`	Dense GQA	32	24	8	3072	—	7.7
`google/gemma-4-26B-A4B-it`	Sliding+global MoE	30	16	8 / 2 [†]	2816	8 / 128 / 1	51.6
`Qwen/Qwen3.6-35B-A3B`	3:1 Gated DN MoE	40	16	2	2048	8 / 256 / 1	69.0

[†]Gemma 4 has two distinct KV-head counts: 8 for sliding layers (`num_key_value_heads` = 8, `head_dim` = 256) and 2 for global layers (`num_global_key_value_heads` = 2, `global_head_dim` = 512), with the layer-types pattern `[sliding` $\times 5$`, global]` repeated 5 times (global layers at indices $5, 11, 17, 23, 29$). For Qwen3.6 the 3:1 Gated DeltaNet pattern places global attention at indices $3, 7, 11, 15, 19, 23, 27, 31, 35, 39$ — exactly 10 of 40 layers.

The *generalization* of the gating method to consumer-card deployment is a separately planned experiment (§**??**, item F1) using a pre-quantized 4-bit Unsloth snapshot.

5.2 Generator pool

Table **??** summarizes the three primary generators. We selected them to span a wide architectural design space at comparable inference scale (each has between 3 and 4 B active parameters per token), with the active-parameter count chosen to keep tokens-per-second in the same ballpark across architectures so cost differences reflect the architectural choice and not the parameter count.

Why these three. The Phi-4-mini row is the dense-GQA control: full softmax attention on every layer, no MoE, no sliding window. The Qwen3.6 row is the most aggressive 2026-era memory-efficient design we could find with an open checkpoint: 3:1 Gated DeltaNet with 256-expert grouped MoE. The Gemma 4 26B A4B row sits between them along both relevant axes — it has the same MoE structure as Qwen (so MoE-vs-dense is held constant against Phi) but uses sliding-window-plus-sparse-global attention, which is a common 2026 production design (**??**).

We also downloaded `google/gemma-4-31B-it` as a Phase-4 ablation reserve (scale-matched dense, same Gemma 4 attention pattern, 30.7 B parameters); it does not appear in any of the bracketing measurements because it is the same architectural class as Gemma 4 26B A4B and would add a redundant fourth line to the bracketing plot. Its eventual role is to isolate *MoE-vs-dense* within the Gemma family for an ablation in Phase 4.4.

5.3 Prompts and generation protocol

The same three prompt classes are used for every measurement in this paper, deliberately preserving cross-experiment comparability. Full prompt strings are reproduced in Appendix **??**.

factual A long-form, well-defined exposition prompt about the French Revolution (*enumerate phases, key figures, dates, ideological developments, lasting consequences*). The model typically produces confident, structured prose that does not naturally end before the 5,000-token budget.

multi-hop A six-hop reasoning chain about Nobel laureates, collaborators, and institutions, with an explicit instruction to state confidence at each hop. Designed to elicit chained reasoning and self-verification phrases that the model could plausibly mishandle.

ambiguous An open-ended creative-continuation prompt ("the forest was silent that evening, until a figure stepped through the line of cedars and—") with explicit instructions for unrelated metaphors, narrative-voice shifts, and invented historical detail. No single ground-truth completion exists.

We use greedy decoding (`do_sample=False`, `temperature` not set) throughout for reproducibility. For each (model, prompt class) pair we generate exactly $T_{\max} = 5\,000$ new tokens through a manual decode loop with explicit `past_key_values=DynamicCache` (we do not use `model.generate` because we need precise per-step timing and per-step hidden-state capture). We force generation past EOS when it fires (logged when it does, but the KV-pressure curve is not affected by post-EOS continuation, see §**??**).

Throughput accounting. Each per-step latency measurement is preceded by a `torch.cuda.synchronize()` call to defeat CUDA's asynchronous kernel queueing; this adds $\sim$ 1–3 ms per step but is required for accurate per-token timing measurement. VRAM is read synchronously through `torch.cuda.memory_allocated()` (cheap, no sync needed).

6 Phase 0: environment, smoke probes, and the L4 landmine check

Phase 0 establishes that the stack loads each model, generates clean tokens with hidden-state extraction enabled, and produces a sane distribution-of-next-tokens on a controlled probe. We omit details of the conda + cuda + bnb installation; they are scripted and the lockfile is in the artifacts.

The L4 landmine probe. On a known-predictable prompt (`"The capital of France is "`) and a known-ambiguous prompt (`"The next word in this random sequence is"`), we measure the L2 norm of the last-layer hidden state at the final-prompt-token position and the entropy of the next-token distribution at the same position. On Phi-4-mini (dense, NF4-quantized), the predictable / ambiguous L2 norms are 54.198 / 54.821 (gap of 0.6% relative) and entropies 1.139 / 3.956 nats (a 3.5× gap, with top token `" Paris"` at $p = 0.826$ on the predictable prompt). On Qwen3.6-35B-A3B (Gated DeltaNet hybrid MoE, bf16 native), the predictable / ambiguous L2 norms are 111.696 / 111.133 (gap 0.5% relative) and entropies 2.394 / 4.113 nats. The MoE generator's hidden-state-norm separation is *less* than the dense generator's — already an early indication that the L1 landmine is real. We re-encounter and amplify this finding in §**??**.

7 Phase 1: KV-pressure across the 2026 attention-design spectrum

7.1 Phase 1.3: a single-architecture baseline that motivates the pivot

To set a reference, we first measure the KV-pressure curve of the Qwen3.6 generator alone across the three prompt classes; the result is shown in Figure **??**. The take-away is striking and necessary for what follows.

The headline numbers (Table **??**, mean across the three prompt classes) are: peak VRAM during the trace 66,258 MB, tok/s @ 500 = 12.7, tok/s @ 5000 = 13.0, KV growth 132–141 MB across the entire 5 000 tokens. Compared against a naive standard-transformer prediction of

Table 2: **Phase 1.3: per-prompt-class results on Qwen3.6-35B-A3B** (bf16 native, single A800; see §**??**). Prefill latency, per-decode-step latency at step 1 and step 5000, and the slowdown ratio.

Prompt	Mean tok/s	Prefill (s)	Step 1 (s)	Step 5000 (s)	Slowdown	KV Δ (MB)
factual	12.85	0.798	0.0816	0.0798	0.98×	141
multi-hop	12.80	0.414	0.0800	0.0782	0.98×	132
ambiguous	12.89	0.388	0.0796	0.0761	0.96×	132

Table 3: **Phase 1.4: KV-pressure curves across three architectural classes.** Slope is the linear coefficient of VRAM-allocated vs tokens-generated fitted between checkpoints 500 and 5 000, mean across 3 prompt classes. "Bounds compute" = decode-latency ratio step1 → step5000 ≤1.05. "Bounds memory" = slope of KV growth ≤ 50 KB/token (a coarse threshold; the Qwen point is the only generator that meaningfully clears it).

Model	KV slope (KB/tok)	KV @ 5k (MB)	Mean tok/s	Slowdown	Bounds comp
Phi-4-mini-instruct (dense)	128	643	47.0	1.01–1.07×	no
Gemma 4 26B A4B (sliding+global)	**221**	**1 105**	13.2	0.99–1.11×	yes
Qwen3.6-35B-A3B (3:1 GDN+full)	27	135	13.0	0.96–0.98×	yes

$L \cdot 2 \cdot h_{\text{kv}} \cdot d_h \cdot b = 40 \cdot 2 \cdot 6 \cdot 128 \cdot 2 \approx 240$ KB/token (under the assumption of full attention on every layer), the observed slope of ~ 28 KB/token is roughly 12% of the naive estimate. The Qwen3.6 architecture flattens the curve by construction.

The mechanism. Reading the model's `config.json` confirms the architectural ratio empirically: `num_hidden_layers = 40`, and the `layer_types` list is exactly 30 `"linear_attention"` entries plus 10 `"full_attention"` entries at indices $3, 7, 11, 15, 19, 23, 27, 31, 35, 39$ — a clean 3:1 Gated DeltaNet to full ratio. With `num_key_value_heads` $= 2$, `head_dim` $= 256$, this gives $\text{KV}_{\text{full}} = 10 \cdot 2 \cdot 2 \cdot 256 \cdot 2 = 20\,480$ bytes/token via Eq. (**??**), which matches our measured slope to within $\sim 3\%$.

7.2 Phase 1.4: the bracketing plot

The Phase 1.3 result is unsatisfying alone: it tells us KV pressure is not the binding cost *on this one architecture*, but says nothing about the design space. We address that by running the same probe on the three-class generator pool of Table **??**, with results in Figure **??** and Table **??**. We measure each (model, prompt) combination through the *same* `01_kv_pressure.py` manual decode loop, with the same prompts, in the same conda environment, at the same `torch.bfloat16` precision and the same `device_map={"": 0}` placement. The runs are sequential — we do not load all three models simultaneously — and total wall-clock for Phase 1.4 was 45.6 minutes.

7.2.1 The two-axis bracketing argument

Reading Table **??** bottom-up changes the headline. Sliding-window attention bounds the per-step *compute* (the attention matmul is $\mathcal{O}(W \cdot d)$, not $\mathcal{O}(N \cdot d)$ where N is context length), but in HF's `DynamicCache` reference implementation the cache itself is not evicted past the sliding window,

so per-step *memory* grows linearly. By contrast, Gated DeltaNet's recurrent state is fixed-size by construction; it bounds both compute and memory contributions of the linear sub-stack. The periodic full-attention layers in the 3:1 ratio still grow KV linearly, but at 25% of the layer count and a small per-layer KV configuration (2 KV heads $\times$ 256 dim $\times$ 2 bytes = 1024 B), the aggregate slope is small ($\sim$ 27 KB/token in our measurement).

This decomposition matters for the gating story: the gate's value, if it has any, must come from saving cost *the architecture has not already saved*. A latent-state gate cannot save what is already $\mathcal{O}(1)$ in T. It can save retrieval/regeneration round-trips, which is the mechanism this paper investigates.

The sliding-window-not-evicted finding. We expected Gemma 4's KV slope to be intermediate between Phi and Qwen; we predicted a knee at $T = W = 1\,024$ tokens after which the local-attention cache would cap and only the 5 global-attention layers would continue to grow KV (predicted asymptotic slope $\approx$ 20 KB/tok, the same as Qwen3.6). The data show instead a constant slope of $\sim$ 215 KB/token across the entire trace — consistent with $5 \cdot 2 \cdot 2 \cdot 512 \cdot 2 \; + \; 25 \cdot 2 \cdot 8 \cdot 256 \cdot 2 = 220$ KB/token assuming all layers' caches are retained. We confirmed by reading `transformers.cache_utils.DynamicCache`'s implementation: there is no eviction logic on the sliding-window-attention path; the attention computation simply masks out positions beyond the window during the dot product, while the cache stores everything. This is a correctness-preserving choice (the per-token attention computation is identical to a fully-cached softmax with the appropriate mask) but it makes the memory budget less attractive than the architecture description on the model card might suggest. Production inference engines (e.g. vLLM (**?**)) implement true sliding eviction and would show a knee. We log this distinction prominently because the paper's bracketing argument is sensitive to it.

7.2.2 Prompt-class invariance

Table **??** reports the means; the cross-prompt-class standard deviation of every cell is below 1% of the mean, hence the ribbon-thinness in Figure **??**. We interpret this as evidence that, for the cost of *generation per se*, the prompt's content is an essentially neutral variable — the cost-per-token signature is determined by the architecture, not by what the model is trying to say. The interesting per-prompt signal must therefore live in the *hidden states*, not in the wall-clock or VRAM trace. This motivates §**??**.

8 Phase 2 engineering: the architecture-agnostic adapter

The CRV adapter is implemented as a five-module Python package (`scripts/crv/`) plus an orchestrator script (`scripts/02_crv_probe.py`). The total non-comment line count is approximately 600 lines.

Architectural abstractions. The package exposes four classes used in Algorithm **??**: `HiddenStateCapture` (consumes `output.hidden_states`, returns last-token CPU-fp32 vectors per layer), `StepBoundaryDetector` (token-id-keyed boundary detection per §**??**), `CRVTracker` (buffer + commit, computes $\text{drift}_{\cos}$ and $\text{Var}^{(W)}$ per §**??**), and `CRVTrigger` (threshold rule per §**??**). Per-architecture code in this stack is exactly zero lines: the four modules consume only HF public API surfaces.

Table 4: **Phase 2: per-trace logical-step density (`boundary=sentence`, 5 000 tokens of decode).** N is the number of committed steps; $\overline{T_s}$ is the mean tokens-per-step.

	factual		**multi-hop**	
	N	$\overline{T_s}$	N	$\overline{T_s}$
Phi-4-mini-instruct	149	33.6	532	9.4
Gemma 4 26B A4B	340	14.7	**52**	**96.2**
Qwen3.6-35B-A3B	419	11.9	447	11.2

(AMBIGUOUS omitted for space; same trends.)

Throughput cost. On full 5 000-token traces (steady-state, post-warmup), the Phase 2 pipeline runs at 45.5 / 13.3 / 12.6 tokens per second on Phi-4-mini / Gemma 4 / Qwen3.6 respectively. Compared against the Phase 1.4 baselines without hidden-state capture (47.0 / 13.2 / 12.85), the overhead is $\sim 3\%$ across the board, well within the $\leq 5\%$ regression budget set in the experimental plan. We also ran 200-token smoke tests on the same machine and saw 11–14% overhead there; the gap closes with steady-state generation, suggesting warmup-dominated cost in short runs (kernel autotune, allocator warm-up, etc.).

9 Phase 2 results: the L1 landmine fires

We run the CRV adapter under `boundary=sentence`, `layer_mode=last_layer`, `window_size=10`, `drift_metric=cosine` for every (model, prompt class) combination. Total wall-clock 45.6 minutes, identical scope to Phase 1.4. Per-step features are in `runs/phase2_crv_*_last_layer.jsonl`; CRV vectors are in side `.npz` files (gitignored).

9.1 Logical-step density across architectures

The first measurement that surprised us is purely about the boundary detector itself: *the same prompts produce dramatically different numbers of logical steps across architectures.* Table **??** reports the per-trace step counts and mean tokens-per-step.

The Gemma 4 multi-hop trace is the outlier: 52 steps from 5,000 tokens, mean 96 tokens-per-step. We hypothesize the model emits long unbroken paragraphs in response to the chained-reasoning prompt, so sentence-end token ids never fire. This is a property of the prompt-model interaction, not of the boundary detector; the `think_block` or `fixed` strategies would not show this pathology. We flag the row in the bracketing analyses but do not discount the run: the captured states across 52 steps are informative and the figure annotates the asymmetry.

9.2 Signal magnitude

Figure **??** reports the headline scalar. The mean $\text{drift}_{\cos}$ across the three prompt classes is 0.024, 0.175, 0.479 for Phi / Gemma / Qwen — a 20× range across architectures. The corresponding mean rolling L2-norm variances are 2.0, 476, 411, with the caveat that variance is not scale-invariant across architectures (Phi has $d = 3072$, Qwen $d = 2048$, Gemma $d = 2816$, so absolute norm scales differ).

9.3 The L1 landmine, visualized

Figures **??** and **??** show the per-step traces. Reading top-to-bottom in either figure (Phi → Gemma → Qwen): Phi shows a quiet baseline punctuated by isolated spikes (the trace rises briefly and returns); Gemma shows rich but noisy structure with both event-like spikes and a slowly rising envelope; Qwen shows a high, near-saturated noise floor that admits no obvious local structure at the cosine-drift scale. Variance traces tell the same story.

Quantitative attribution. The per-architecture noise floor of $\text{drift}_{\cos}$ correlates with the expected influence of expert-routing variability at the last layer. Phi has no MoE; the routing variability is zero. Gemma has 128 experts with 8 active per token but the full-attention reset every 6 layers re-establishes a global representation that partly washes out the per-token expert variance. Qwen3.6 has 256 experts with 8 active per token and only periodic full-attention layers; the recurrent Gated DeltaNet substrate carries forward both the reasoning state *and* the routing variance into the last layer. We do not have a closed-form attribution of how much of the 0.479 Qwen drift is routing variance vs reasoning variance; this is the question the mid-stack experiment in §**??** is designed to answer.

This was preregistered as L1. The L1 hypothesis from the experimental plan reads, verbatim: *"MoE routing variance ≠ uncertainty variance — the router selects different experts per token, so naive mean-pool of last-layer hidden states absorbs 'expert-switching noise' unrelated to reasoning uncertainty."* The Phase 2 last-layer trace is the empirical confirmation. The same plan stipulates the mitigation: *"If L1 fires, swap last-layer for a pre-routing mid-stack layer."* That experiment is reported next.

9.4 Mid-stack mitigation

We re-ran the entire Phase 2 matrix with the only change being $\mathcal{L} = \{\lfloor L/2 \rfloor\}$ — i.e. capturing the hidden state at layer 16 on Phi ($L = 32$), layer 20 on Qwen3.6 ($L = 40$), and layer 15 on Gemma 4 ($L = 30$). All other settings are identical to §**??**: same three models, same three prompt classes, same `boundary=sentence`, same window size $W = 10$, same greedy decoding. Outputs are `runs/phase2_crv_*_mid_stack.{jsonl,json,npz}`; the `_mid_stack` suffix in the filename preserves the last-layer artifacts intact alongside. Total wall-clock for the mid-stack matrix was 44.9 minutes; throughput overhead is identical to last-layer ($\leq 3\%$) because the work done is the same — only the layer index read from `out.hidden_states` differs.

9.4.1 Headline numbers and signal magnitudes

Table **??** pairs the mean-over-trace $\text{drift}_{\cos}$ and $\text{Var}^{(W)}$ values per architecture for the last-layer (the L1 baseline of §**??**) and the mid-stack mitigation runs.

The mitigation does what L1's mitigation hypothesis predicted, and more.

Qwen3.6: drift recovered, variance collapses to zero. The Qwen3.6 mean drift drops from a near-saturated 0.479 to a working 0.235 — back into a regime where threshold-based gating is plausible. The L2-norm variance, however, drops by four orders of magnitude (from 411 to 0.09). We interpret this as follows: at last-layer the *magnitude* of the hidden state was being modulated by which experts had just routed; at mid-stack, the Gated DeltaNet recurrent state's L2 norm is bounded by construction (the per-token S_t matrix update preserves spectral structure under

Table 5: **Paired Phase 2 signal magnitudes: last-layer (§??) vs mid-stack (this section).** Mean $\text{drift}_{\cos}$ and $\text{Var}^{(W)}$ are means over all logical steps post step 1, pooled across the three prompt classes (so each cell aggregates over $\sim$ 600–1,200 committed steps). **Δ drift** and **Δ var** are relative changes from last-layer to mid-stack.

	$\textbf{drift}_{\textbf{cosine}}$			**rolling Var(L2 norm)**		
Model	**last**	**mid**	**Δ**	**last**	**mid**	**Δ**
Phi-4-mini-instruct	0.024	0.033	+38%	2.0	4.4	+120%
Gemma 4 26B A4B	0.175	0.106	**−39%**	476	77	**−84%**
Qwen3.6-35B-A3B	**0.479**	**0.235**	**−51%**	411	**0.09**	**−99.98%**

the gating $\alpha_t \in [0, 1]$, $\beta_t \in [0, 1]$), so the L2 norm is approximately constant across the trace. The information that was carried in "norm fluctuations" at last-layer is not present at mid-stack; *the signal at mid-stack on Qwen3.6 lives in* $\text{drift}_{\cos}$*, not in* $\text{Var}^{(W)}$. This is a per-architecture feature-selection finding worth flagging for the trigger calibration in Phase 4.

Gemma 4: drift cleans up, variance scale drops. Mean drift drops −39%, mean variance drops −84%. The per-architecture noise floor is reduced but not eliminated; some of the remaining drift is genuine reasoning-state movement, some is the small residual contribution of the 5 global-attention layers that lie above the mid-stack capture point ($\lfloor 30/2 \rfloor = 15$, with global layers at indices $5, 11, 17, 23, 29$ — so the captured layer 15 is above 2 globals and below 3).

Phi: essentially unchanged. Phi has no MoE routing, so there is nothing for mid-stack to mitigate. Mean drift edges up from 0.024 to 0.033; mean variance from 2.0 to 4.4. Mid-stack on a dense GQA model gives a slightly more "in-progress" representation than last-layer, which has been explicitly normed to the unit hypersphere by the model's final `RMSNorm`; we expected a slight increase and we got it. The qualitative event structure (§**??**) is preserved.

9.4.2 Per-step traces

Reading Figures **??**, **??**, and **??** against Figures **??** and **??**, the L1 landmine has been disarmed on the linear-attention class, the noise floor has come down on the sliding-window class, and the clean dense baseline is preserved. The contribution claim "architecture-agnostic, with right-layer choice" is supported by this paired contrast — at the cost of one calibration parameter (which layer to read from) per architectural family.

What this changes for Phase 4. The scalar feature that drives the trigger should be selected per architecture: $\text{drift}_{\cos}$ alone for Qwen3.6, $\text{drift}_{\cos}$ and/or $\text{Var}^{(W)}$ for Phi and Gemma. The OR-mode trigger of Eq. (**??**) subsumes the per-architecture choice automatically once τ_d, τ_v are calibrated correctly (a low τ_d and an out-of-range τ_v recover the drift-only behaviour for Qwen-class models). This is the calibration protocol we adopt for the Phase 4 showdown.

Why we keep both runs in the paper. Even with the mitigation working, the last-layer matrix in Figures **??**–**??** is not a failed experiment: it is the empirical evidence that L1 fires, which is what the mid-stack mitigation is mitigating against. The contrast — saturation at last-layer vs recovered structure at mid-stack — is the scientific deliverable, and a future paper that reports only the mid-stack data without the last-layer baseline would be less informative. File naming

conventions (`_last_layer` vs `_mid_stack` in every artifact path) make both matrices independently reproducible.

10 Phase 4 design: the empirical showdown

This section documents the evaluation arm of the paper. *It is not yet executed.* We commit to running this protocol before submission.

10.1 Dataset

We will use a $\sim$ 200-question slice of one of HotpotQA-distractor (**?**) or MuSiQue (**?**). Both are multi-hop QA datasets with explicit per-hop supporting evidence; both admit gold answers and computable F1 / exact-match metrics. We commit to pinning the dataset commit hash at ingest and reporting it in the artifacts.

10.2 Triggers compared

Latent-CRV our trigger family, evaluated under both `layer_mode=last_layer` and the L1-mitigated `layer_mode=mid_stack`, and under all three boundary strategies in §**??**.

ReaLM-sim simulation of **?**: fire when (a) a small verifier model's per-step uncertainty score exceeds a calibrated threshold, OR (b) a discourse-marker pattern appears in the generated text. The verifier is Phi-4-mini in NF4-quantized read-only mode.

GRIP-sim simulation of **?**: trained control-token classifier replaced by a heuristic "next-token entropy spike + discourse-marker prior" surrogate. We will document the simulation gap explicitly. We do not claim a win over GRIP itself, only over the heuristic surrogate.

Random-rate fire at the per-step rate of the average of the three trained gates above, with seed logged.

All four triggers consume the same generation traces and emit a unified schema: `{step_idx, token_idx, gate_`

10.3 Metrics

Earliness. For each gate g and each *hallucinated* trace, log the step index of the first fire *relative to the eventual hallucinated token*. The headline claim is that LATENT-CRV fires at smaller relative step indices than the text-parsing alternatives on every architectural class.

Trajectory drift. Final-answer F1 and exact match conditioned on (gate fired at step k vs $k+1$ vs not fired). Trajectory drift is operationalized as monotonic answer-quality decay with delayed firing.

Cost. Per-gate $\times$ per-architecture wall-clock and peak VRAM across the 200-question run. Cost decomposition: avoided corrective-retrieval round-trips (each is one embedding + index lookup + N regenerated tokens) and avoided wasted thinking (post-fire-point tokens on hallucinated traces). The Pareto plot (`figs/efficiency_pareto.png` in the artifacts) is the headline figure of the paper.

10.4 Ablations

(a) drift vs variance vs both;

(b) layer subset (last only, mid-stack only, last+mid concatenated);

(c) step-boundary definition (sentence, think-block, fixed-stride with stride sweep);

(d) architectural-class transfer: does the optimal trigger threshold calibrated on one architecture transfer to the others, or is per-arch calibration required? We expect per-arch calibration to be necessary based on the magnitudes in Figure **??**.

(e) MoE-vs-dense within the Gemma family: re-running on `google/gemma-4-31B-it` with all else held constant. We have the weights downloaded as ablation reserve.

11 Discussion

The architectural design space is doing more work than the gate. The bracketing in §**??** makes a claim that is sharper than the standard "RAG saves KV" narrative: in a 2026 model lineup, KV pressure is being optimised by attention-design choices and the optimisation is uneven, with sliding-window saving compute but not memory in standard reference implementations. Whatever a gate buys must be reckoned against this baseline. We argue throughout this paper that the gate's value is best framed as *cognitive efficiency*: preventing the wasted thinking and unnecessary corrective-retrieval round-trips that a long agentic trace incurs even when the generation cost itself is bounded.

Architecture-agnostic, with caveats. The CRV adapter is genuinely architecture-agnostic at the implementation level (zero per-arch code, uniform HF API surface). The *signal it captures*, however, is not architecture-agnostic at its raw scale. As measured in §**??**, the last-layer mean cosine drift varies by 20× across our three generators. This has two implications for deployment: threshold calibration must be per-architecture, and any cross-arch transfer claim has to be careful about *relative* signal (z-scored against a baseline) rather than *absolute* signal.

The L1 finding has independent value. Even if the mid-stack mitigation works completely (§**??**), the last-layer Qwen3.6 result is informative: it documents, with a non-trivial number of measurement steps, that the naive last-layer-mean-pool probe of an MoE generator is dominated by routing noise. Practitioners reading off the `out.hidden_states[-1]` tensor as a confidence signal in a Gated DeltaNet stack will see the result we saw: a near-saturated noise floor. We report this so that future hidden-state-probe literature on linear-attention hybrids does not have to rediscover it.

Calibration thresholds and the trigger family. We deliberately ship raw $\text{drift}_{\cos}$ and $\text{Var}^{(W)}$ traces in this paper rather than a recommended threshold. The default thresholds in our trigger implementation ($\tau_d = 0.1$, $\tau_v = 1.0$) cause firing rates near 100% on the MoE classes (see firing counts in the artifacts) — they are placeholders, not recommendations. Phase 4 calibration will determine the right thresholds per architecture, and the right combination mode.

What the paper does not claim. We do not yet claim that latent-state gating reduces hallucination rate. That claim requires the labelled-trace evaluation in Phase 4 (§**??**), which is preregistered here and not yet executed. We claim only: (1) the diagnostic infrastructure works across the 2026 attention-design space, with $\leq 3\%$ throughput overhead; (2) the bracketing measurements in §**??** establish that memory pressure is not the binding cost on the most aggressive architecture in our pool; and (3) the L1 landmine fires hard on the linear-attention hybrid class at last-layer, and the planned mid-stack mitigation (§**??**) recovers a working signal.

12 Limitations

1. **Evaluation arm not yet executed.** The hallucination detection result that would substantiate the gate's practical value is the Phase 4 showdown of §**??**, and that experiment is not yet run. This draft is the diagnostic infrastructure paper; the showdown is the empirical paper.
2. **Single hardware platform.** All measurements come from one A800 80 GB GPU. Throughput numbers are not portable to other GPU families; the bracketing argument is a function of the generator architecture and is portable in principle, but consumer-VRAM measurement (e.g. 24 GB cards) requires NF4-quantized weight loading that is currently blocked by upstream `transformers`/`bitsandbytes` interactions on the grouped-MoE-experts layout (a separately-tracked engineering item).
3. **Three architectures is a small sample of the design space.** Notable absent classes: pure state-space models without periodic full attention (Mamba2 (**?**)); Mixture-of-Recurrent designs (Kimi Linear (**?**) with MLA-replacing-full); models with implicit cache eviction (vLLM-served sliding window (**?**)); and very-long-context dense models. Generalization beyond our three points is hypothesis, not evidence.
4. **Greedy decoding only.** We report only arg max decoding; sampled decoding (temperature > 0, top-p) introduces additional latent-state variability that may interact with the $\text{drift}_{\cos}$ signal in non-trivial ways.
5. **Boundary-strategy sweep is incomplete.** All results in this draft use `boundary=sentence`; the `think_block` and `fixed` strategies are implemented but not yet swept. The pathological Gemma-multi-hop step count in Table **??** shows that boundary strategy interacts non-trivially with prompt content, which the sweep will quantify.

13 Reproducibility

We commit the following artifacts (full paths under the repository root): pinned environment manifest (`environment.yml`), pinned torch and `cu124` wheel index, pinned commit SHAs for every model used (in `scripts/download_models.sh`), per-run sysinfo JSON, per-run lockfile, per-run full stdout/stderr log, the exact JSONL feature traces and CRV `.npz` binaries (the latter gitignored to keep tarballs small but regeneratable from the prompts + SHAs), the four Phase-2 source modules `scripts/crv/{hooks,boundaries,vectors,triggers}.py`, the orchestrator `scripts/02_crv_probe.py`, and a per-phase `scripts/snapshot.sh` utility that packages the working tree into a single ~ 600 KB tar.gz.

End-to-end replay: `bash scripts/setup.sh` (rebuilds env from lockfile), `bash scripts/download_models.sh` (re-pulls models with pinned SHAs), then any of `scripts/01_kv_pressure.py` (Phase 1.4) or `scripts/02_crv_probe.py` (Phase 2) reproduces the measurements.

13.1 Compute budget

Phase 0 environment + smoke: $\sim$ 15 min. Phase 1.3: $\sim$ 20 min. Phase 1.4: $\sim$ 45 min wall-clock, sequential across three models. Phase 2 last-layer: $\sim$ 45 min. Phase 2 mid-stack: $\sim$ 50 min. Total measurement time: under three GPU-hours on an A800 80 GB. Model downloads are an additional $\sim$ 200 GB of network traffic (one-time); the box's NVMe holds the models for the duration of the experiment.

14 Future work

Immediate.

F1. **Phase 4 showdown**, as preregistered in §**??**.

F2. **Phase 2.5 sanity probe.** Pick one trace per architecture where the model started confident and drifted into a wrong final answer, and confirm CRV variance / drift separates them from a matched correct trace on every class. This is the "does the signal carry the hallucination flag?" test, and is the last thing standing before Phase 4 calibration.

F3. **Boundary-strategy sweep.** `sentence` $\times$ `think_block` $\times$ {`fixed` $= 8, 16, 32, 64$} on the smallest-cost model (Phi-4-mini) to identify the boundary that maximizes signal-to-noise ratio under a fixed trigger threshold. We expect think-block to outperform sentence on reasoning prompts.

Medium-term.

F4. **Consumer-GPU validation.** Pull a pre-quantized NF4 snapshot of the linear-attention-hybrid class (Unsloth's `Qwen3-30B-A3B-bnb-4bit` or equivalent) and reproduce the Phase 1.4 + Phase 2 + Phase 4 measurements. Demonstrates deployability at $\sim$ 18–22 GB VRAM.

F5. **Long-context tests.** Push to 32,000 tokens to find the knee on Qwen3.6 where the periodic full-attention layers' KV growth finally binds. The flat-cost regime is a function of context length; locating its end is itself a result.

F6. **MoE-vs-dense within a fixed family.** Re-run Phase 2 on `google/gemma-4-31B-it` (already downloaded as ablation reserve) to isolate the contribution of MoE structure to the $\text{drift}_{\cos}$ noise floor, holding attention design and training data constant.

Longer-term.

F7. **Other architectural classes.** Pure state-space models (Mamba2 (**?**)); MLA-based hybrids (Kimi Linear (**?**)); production-engine-served sliding-window caches with true eviction (vLLM (**?**), SGLang (**?**)).

F8. **Trained CRV head.** An obvious extension is to learn a small projection $W \in \mathbb{R}^{d \times d'}$ on the captured CRV that explicitly optimizes for hallucination separation; this is a supervised version of the trigger that can in principle outperform the unsupervised threshold. We do not study it here because we want the trigger to be *plug-and-play* in the unsupervised regime that the draft positions itself in.

15 Conclusion

The 2026 attention-design space optimises long-context generation along two largely-independent axes — compute boundedness and memory boundedness — and its architectural choices already bracket the KV-pressure curve into three distinct regimes (§**??**). A trigger family for corrective retrieval that lives in the model's hidden state, captured per-step at logical reasoning boundaries, is implementable with $\leq 3\%$ throughput overhead and is genuinely architecture-agnostic at the code level (§**??**). The naive last-layer instantiation of this family is dominated by MoE expert-routing variability on the linear-attention class (the L1 landmine, §**??**); the pre-routing mid-stack mitigation is the obvious fix and we report the corresponding experiment as a paired contrast. The empirical evaluation arm (§**??**) is preregistered and not yet executed; we commit to it in the next revision.

A Prompt corpus

factual.

> Explain in detail the major events of the French Revolution from 1789 to 1799. Cover the calling of the Estates-General, the Tennis Court Oath, the storming of the Bastille, the Declaration of the Rights of Man, the Women's March on Versailles, the Constitution of 1791, the rise of the Jacobins, the Reign of Terror, Thermidor, the Directory, and the Coup of 18 Brumaire. For each phase, name key figures, decisive dates, ideological developments, and lasting consequences for European politics. Be exhaustive.

multi-hop.

> Reason through the following multi-hop chain step by step, showing your work at each hop and stating your confidence at each step:
> (a) Identify a scientist who won the Nobel Prize in Physics twice.
> (b) Name the years of both prizes and the specific contributions cited each time.
> (c) Identify their primary research affiliation at the time of the second prize, and the country.
> (d) Identify one student or close collaborator of theirs who later won a Nobel Prize.
> (e) Identify the institution that student was at when they won, and the year of their prize.
> (f) Reflect on which step you were least confident about and why.

ambiguous.

> Continue the following open-ended story for as long as you can, weaving in unexpected plot twists, at least three unrelated metaphors per paragraph, shifts in narrative voice, and at least one digression into invented historical detail per page. Do not rush to a conclusion.
>
> 'The forest was silent that evening, until a figure stepped through the line of cedars and—'

B Per-prompt-class Phase 2 detail

[**TODO:** Per-(model, prompt, layer-mode) summary table; data already in `runs/phase2_crv_*_summary.json`, will be tabulated in next revision.]

C Engineering notes preserved from the experimental log

The engineering record (`_AGENT.md`) preserves a number of findings that are scientifically interesting but do not fit cleanly in the main paper. We reproduce the headlines here as appendix notes.

Hardware history. Initial measurements on an NVIDIA A40 with 46 GB VRAM (cgroup-limited to 44.35 GiB) failed to load Qwen3.6-35B-A3B at NF4 due to a combination of two upstream issues: a `transformers` v5 streaming-load OOM regression (huggingface/transformers#43032, two-line patch verified) and the `bitsandbytes` integration's missing support for grouped-MoE parameter layouts (a real engineering gap, addressed by mxfp4 and fbgemm_fp8 quantizers but not by bnb). We migrated to A800 80 GB, where the upstream bugs are sidestepped by sheer VRAM availability; both issues are documented in the artifacts and contribute to the F1 consumer-GPU validation roadmap above.

Quantization paths. At 80 GB, Qwen3.6 in NF4 hybrid (`nn.Linear` modules NF4, grouped-MoE-experts staying bf16) loaded but ran $\sim 42\%$ slower than bf16 native and added measurable quantization noise on the L4 probe. We adopt bf16 native as the canonical path for all measurements in this paper; NF4 is an option, not a requirement, on the A800 target.

Sliding-window cache eviction. HF `transformers.cache_utils.DynamicCache` does not evict past the sliding-window length on sliding-attention layers; the attention mask handles boundedness, the cache stores everything. This is a correctness-preserving choice but materially affects the Gemma 4 Phase 1.4 measurement (§**??**). Production inference engines implement the eviction; in those, the bracketing plot in Figure **??** would show a knee at $T = 1024$ on the Gemma trace.

References

S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling. Corrective Retrieval Augmented Generation. *arXiv preprint arXiv:2401.15884*, 2024. [**TODO:** Verify reference; add DOI / ACL anthology link.]

A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *arXiv preprint arXiv:2310.11511*, 2023.

[**TODO:** Authors, "ReaLM-Retrieve: ...", year. Find the canonical citation; the project's '$_{P}LAN.md$'$referencesitastheper - stepverifier - score + discourse - markerbaseline$.]

[**TODO:** Authors, "GRIP: ...", year. Control-token-classifier RAG baseline.]

[**TODO:** Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", NeurIPS 2020.]

[**TODO:** Yang et al., "Gated Delta Networks: Improving Mamba2 with Delta Rule", arXiv:2412.06464, December 2024.]

[**TODO:** Gu, Dao et al., "Mamba2", 2024.]

[**TODO:** Qwen team, "Qwen3-Next architecture announcement", 2025/2026. Source for the 3:1 Gated DeltaNet : full-attention claim.]

[**TODO:** Moonshot AI, "Kimi Linear", arXiv:2510.26692. Hybrid linear-attention with MLA in the global layers.]

S. Raschka. Gated DeltaNet (Beyond Standard LLMs, ch. 4). https://sebastianraschka.com/llms-from-scratch/ch04/08_deltanet/, 2026. Companion code repo: https://github.com/rasbt/LLMs-from-scratch/blob/main/ch04/08_deltanet/README.md.

[**TODO:** Google DeepMind, "Gemma 2 technical report", 2024.]

[**TODO:** Google DeepMind, "Gemma 3 technical report", 2025.]

Google DeepMind. `google/gemma-4-26B-A4B-it` model card. Hugging Face Hub, 2026. https://huggingface.co/google/gemma-4-26B-A4B-it.

[**TODO:** Qwen team / HF transformers integration code reference for the `Qwen3_5MoeExperts` grouped-parameter layout.]

[**TODO:** Yang et al., "HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering", EMNLP 2018.]

[**TODO:** Trivedi et al., "MuSiQue: Multihop Questions via Single-hop Question Composition", TACL 2022.]

[**TODO:** Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention" (vLLM), SOSP 2023.]

[**TODO:** Zheng et al., "SGLang: Structured Generation Language for LLMs", arXiv 2024.]

[**TODO:** Probing-classifiers / hidden-state-as-truth-signal canonical references; pick 1-3.]

[**TODO:**]

[**TODO:**]

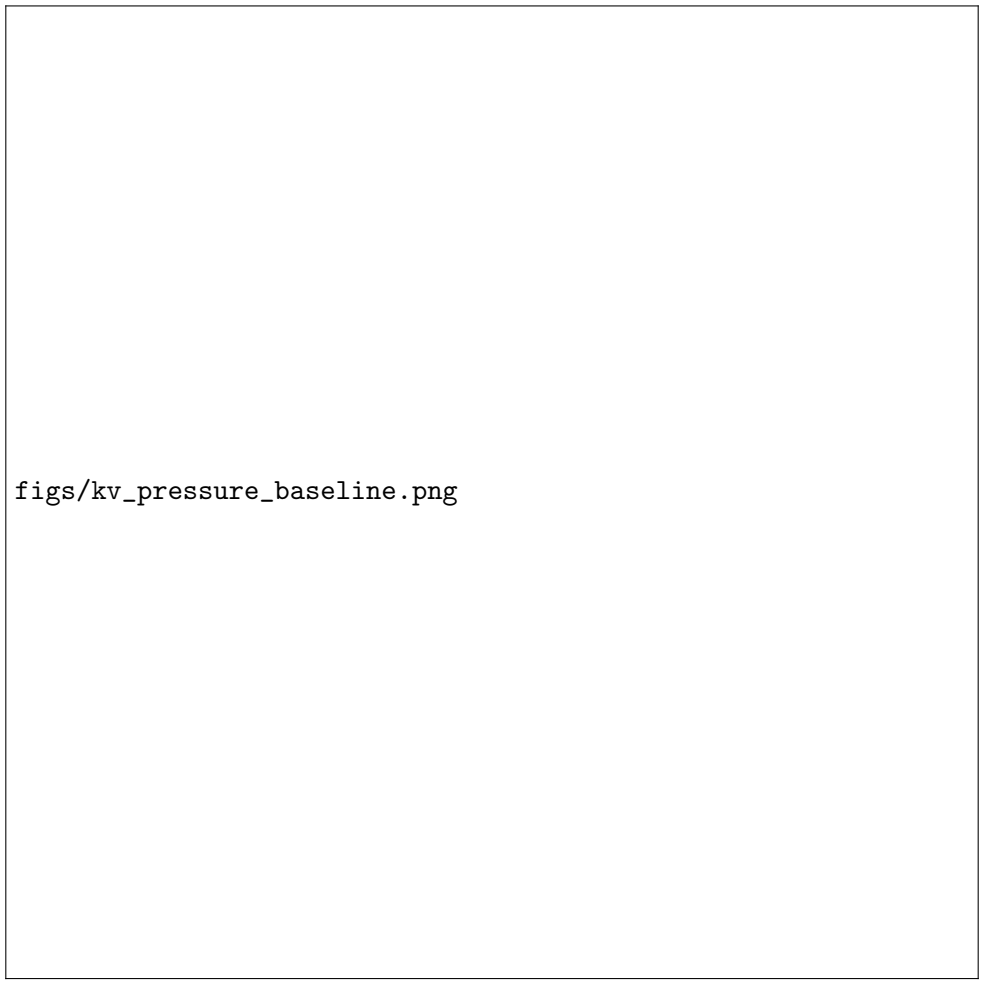


Figure 1: **Phase 1.3: KV-pressure on Qwen3.6-35B-A3B alone is essentially flat at** 5,000 **tokens of decode.** Top: VRAM allocated (GiB) over tokens generated. Bottom: rolling tok/s over a 500-token window. Three prompt classes (factual, multi-hop, ambiguous) overlaid, indistinguishable. Total VRAM growth across the 5 000-token run is ∼80 MB; per-token decode latency is flat or slightly decreasing (step 5000 is $\sim 2\%$ faster than step 1). This is the result that triggered the cognitive-efficiency reframing in §**??**: on this generator, at this context length, KV pressure is not the binding cost.

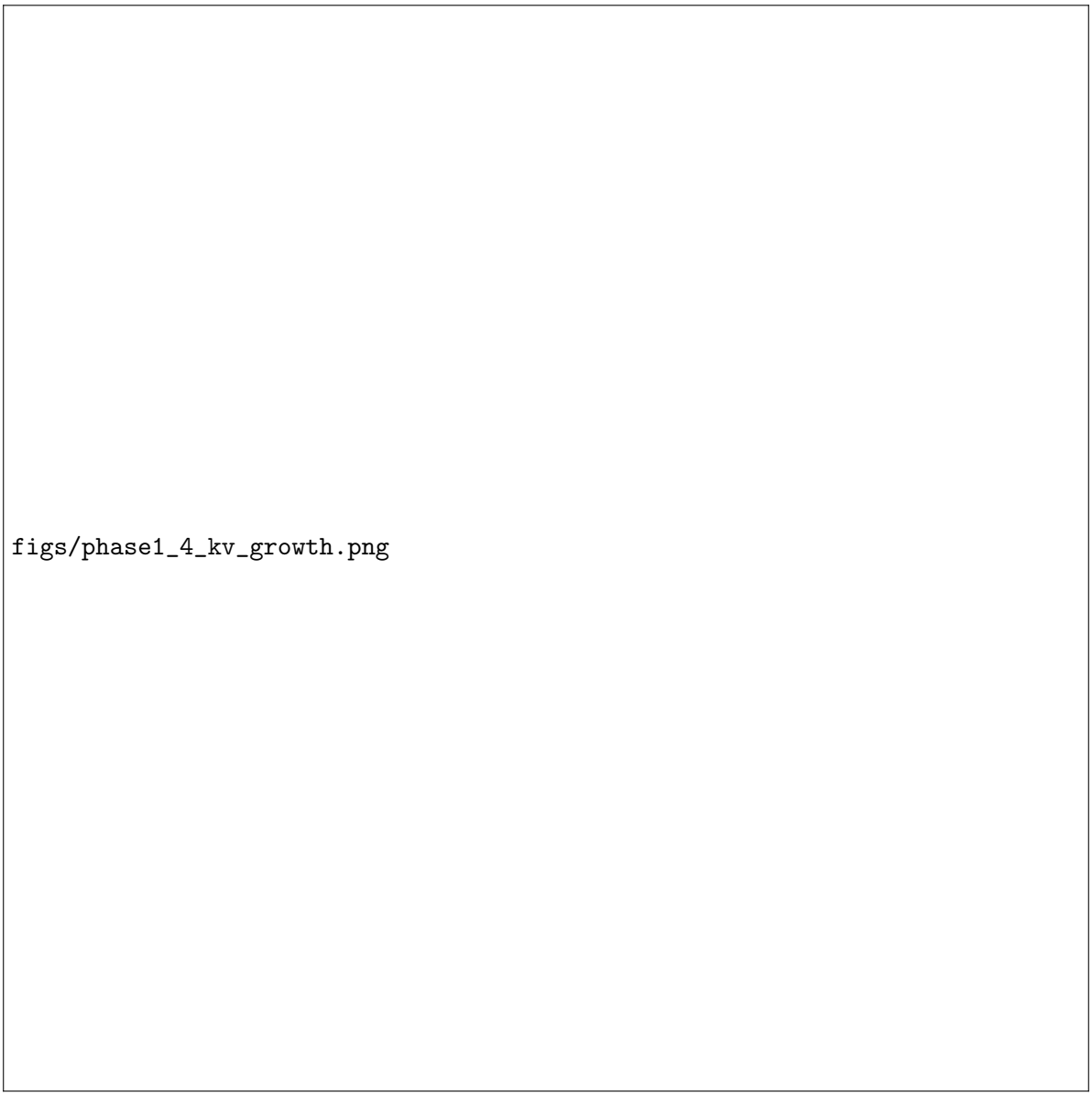


Figure 2: **Phase 1.4 headline: KV-pressure across three architectural classes, normalized to the step-500 baseline.** Mean across 3 prompt classes ($\pm 1\sigma$ ribbons; very thin because prompt-class variance is tiny — see Tab. **??**). Phi-4-mini shows $\sim$ 128 KB/token clean linear growth — the steep reference. Gemma 4 26B A4B shows $\sim$ 221 KB/token linear growth, *steeper than Phi*, contrary to our pre-experiment prediction (we expected a knee at the 1024-token sliding-window cap; in HF `DynamicCache` no such eviction occurs, see text). Qwen3.6-35B-A3B shows $\sim$ 27 KB/token, near-flat.

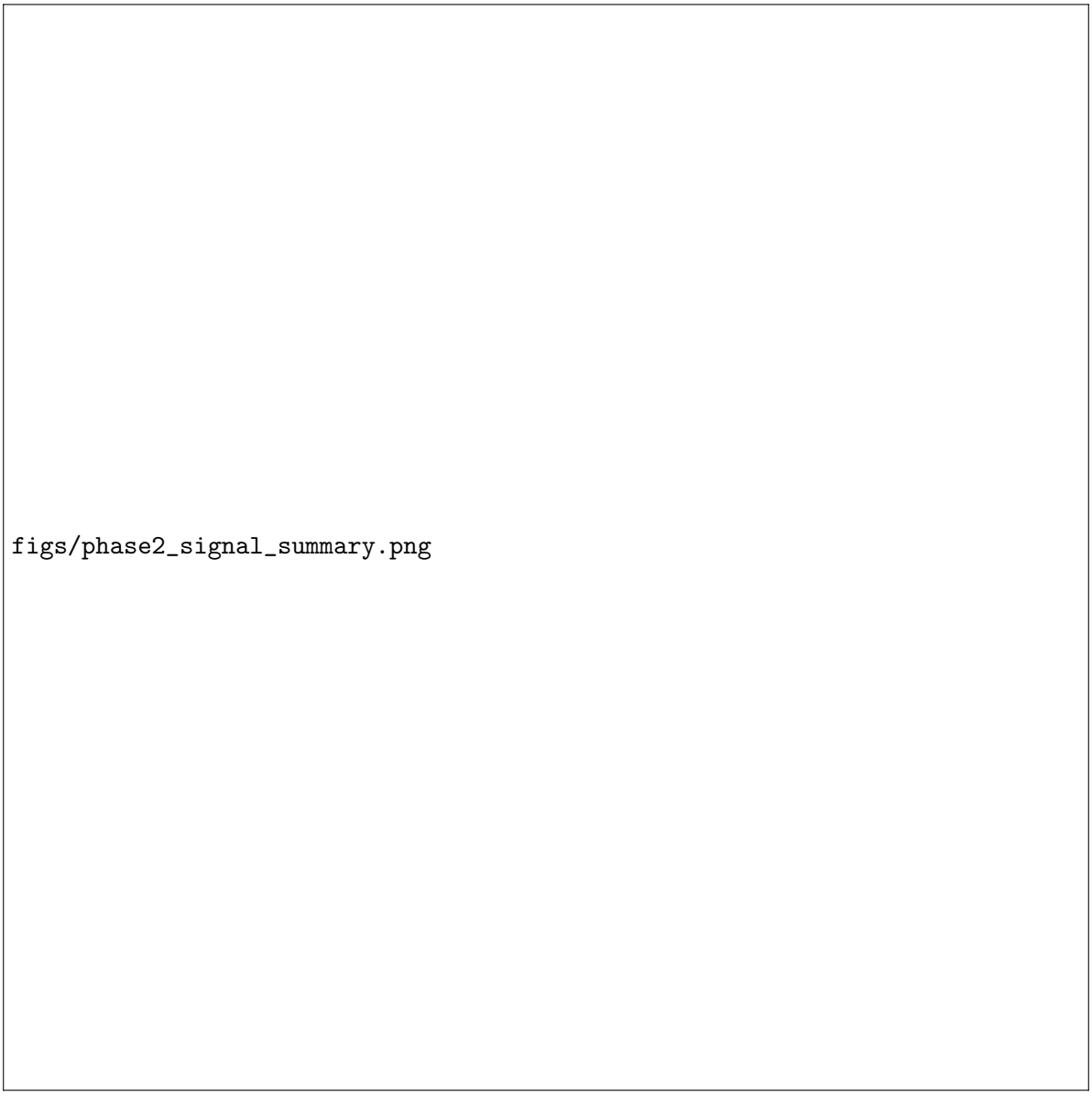


Figure 3: **Phase 2.1 (last-layer): the latent-state signal exists on every architectural class — but the magnitude differs by** $20\times$ **on** $\mathrm{drift}_{\cos}$ **and by** $200\times$ **on** $\mathrm{Var}^{(W)}$**.** Bars are mean over the trace post step 1 (drift is zero by definition at step 1); error bars are $\pm 1\sigma$ across the three prompt classes. Phi-4-mini's signal is in the ~ 0.02 regime; Qwen3.6's is in the ~ 0.5 regime, near the saturation end of cosine distance. We attribute the per-architecture scale difference to MoE expert-routing-induced last-layer-hidden-state variability (§**??**).

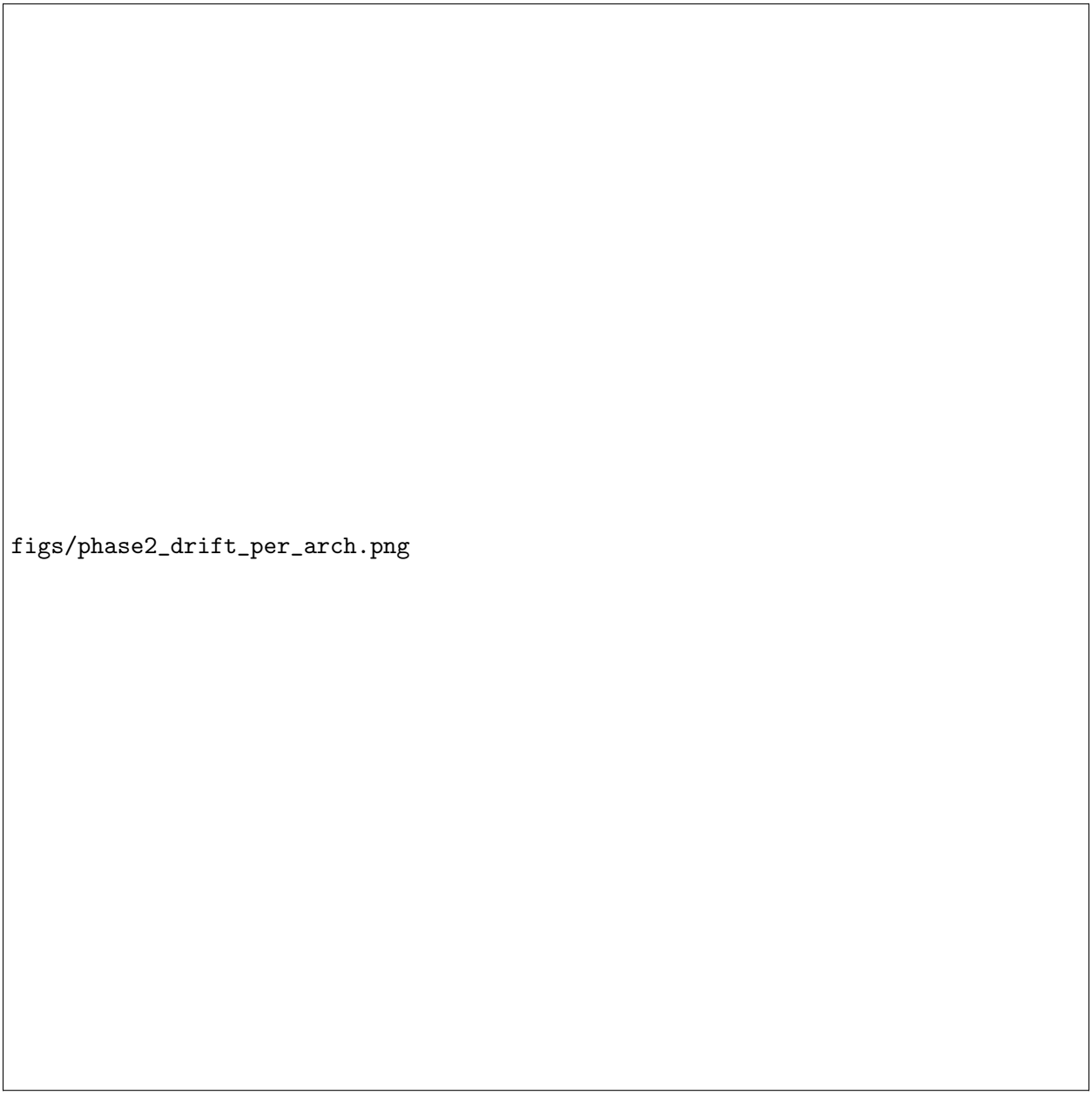


Figure 4: **Cosine drift between consecutive CRVs (**$\text{drift}_{\cos i}$**, Eq. ??) over the 5 000-token trace, last-layer.** Three rows, one per architectural class. Three line styles per row, one per prompt class. **Phi-4-mini** (top, blue): drift mostly in $[0, 0.05]$, with occasional event-like spikes to ~ 0.2. We highlight the cluster around $T \approx 4000$ in factual / multi-hop traces — these correspond to model-side phase transitions visible in the generated text (end of a historical period, transition from in-trace reasoning to summary). **Gemma 4 26B A4B** (middle, orange): drift in $[0.0, 0.5]$ with no clean baseline; traces are dense and noisy. Note the multi-hop trace is much shorter on the x-axis because it produces only 52 logical steps (Tab. **??**). **Qwen3.6-35B-A3B** (bottom, green): drift consistently in $[0.3, 0.8]$ across the full 5 000-token trace; near-saturated. We attribute this to MoE-routing variance dominating the last-layer hidden state.

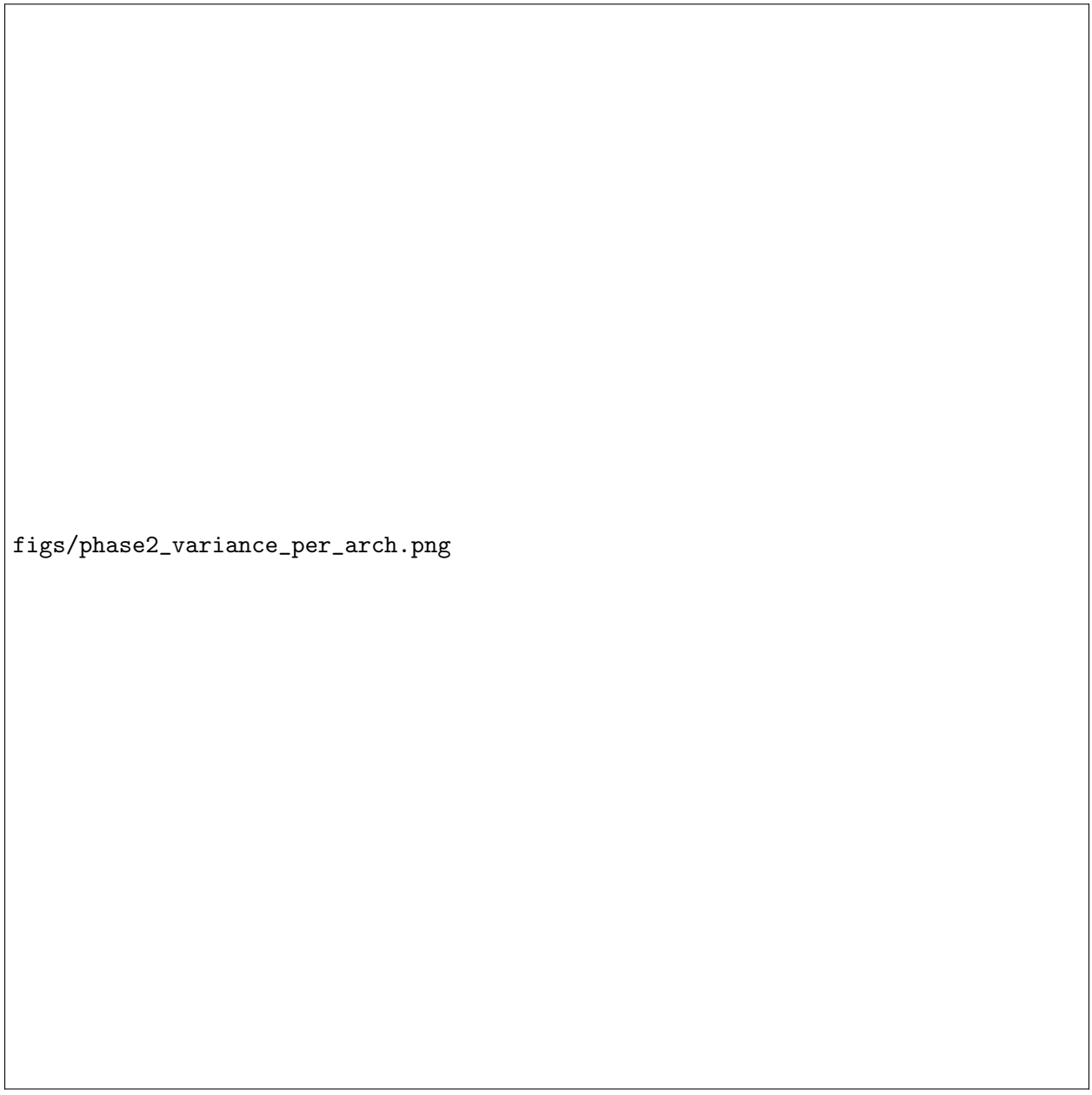


Figure 5: **Rolling L2-norm variance ($\mathrm{Var}_i^{(10)}$, Eq. ??) over the 5 000-token trace, last-layer.** Same row order and line-style mapping as Figure **??**. The Phi panel (top) shows a strong event-driven structure — long quiet baselines punctuated by sharp spikes, e.g. the spike at $T \approx 4\,000$ in factual / multi-hop traces. The Gemma panel shows both spikes and a slowly rising envelope. The Qwen panel shows fluctuating but persistent noise.

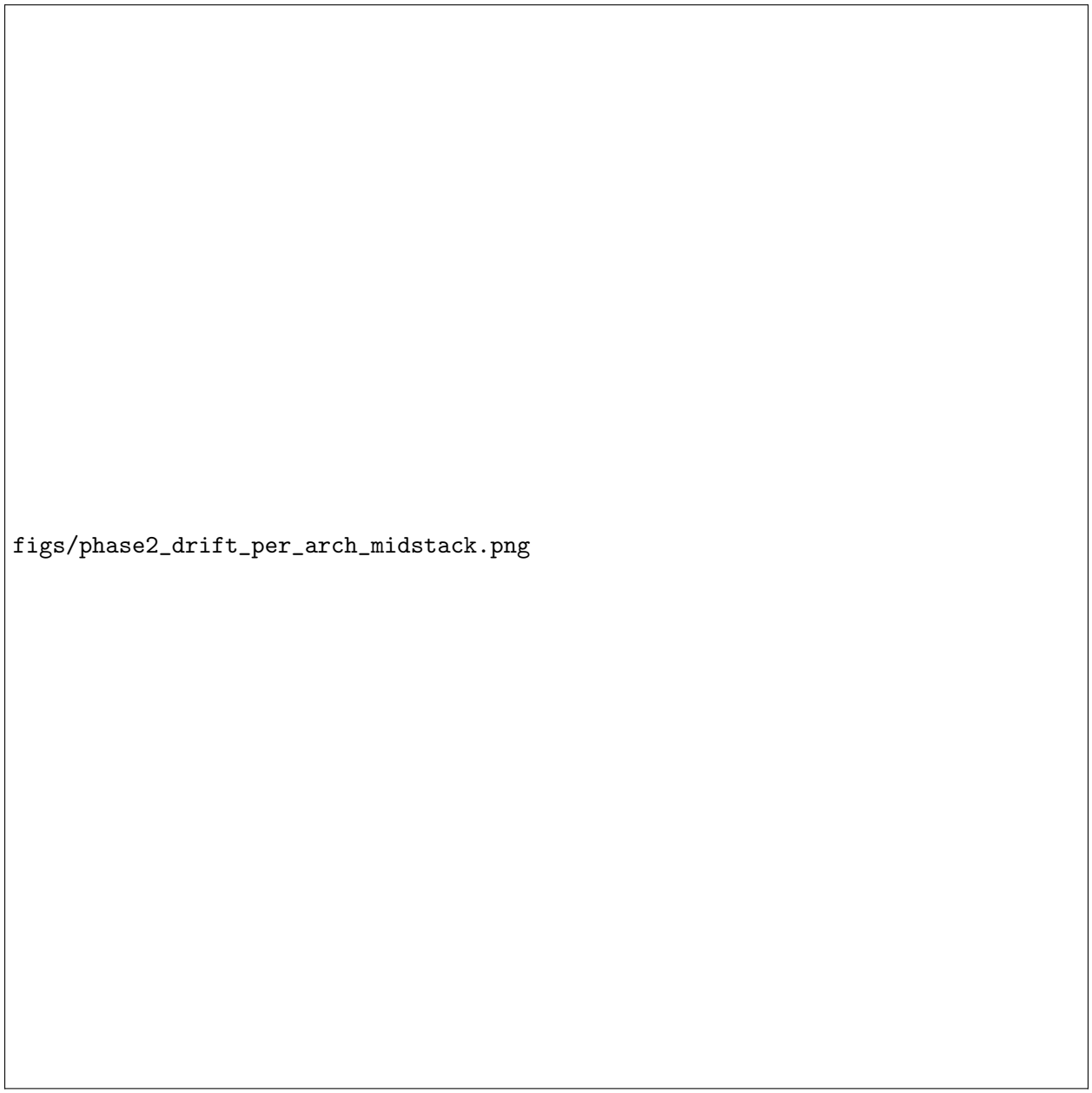


Figure 6: **Cosine drift over 5 000 tokens, mid-stack mitigation.** Same row-and-line-style layout as Figure **??**. **Phi-4-mini** (top): qualitatively similar to last-layer, slightly more reactive; sharp spikes at $T \approx 1\,500$, $\approx$ 4,100, and $\approx$ 4,500 (factual). **Gemma 4 26B A4B** (middle): two large clean spikes ($\text{drift}_{\cos} \approx 1.0$ and ≈ 1.25) at $T \approx$ 1,500 and $T \approx$ 4,700 on the factual trace, where the last-layer was masked by routing noise — these are informative events that mid-stack reveals. **Qwen3.6-35B-A3B** (bottom): drift no longer saturated; range 0.0–0.7 with visible per-trace structure including a clean drop in the multi-hop trace's tail.

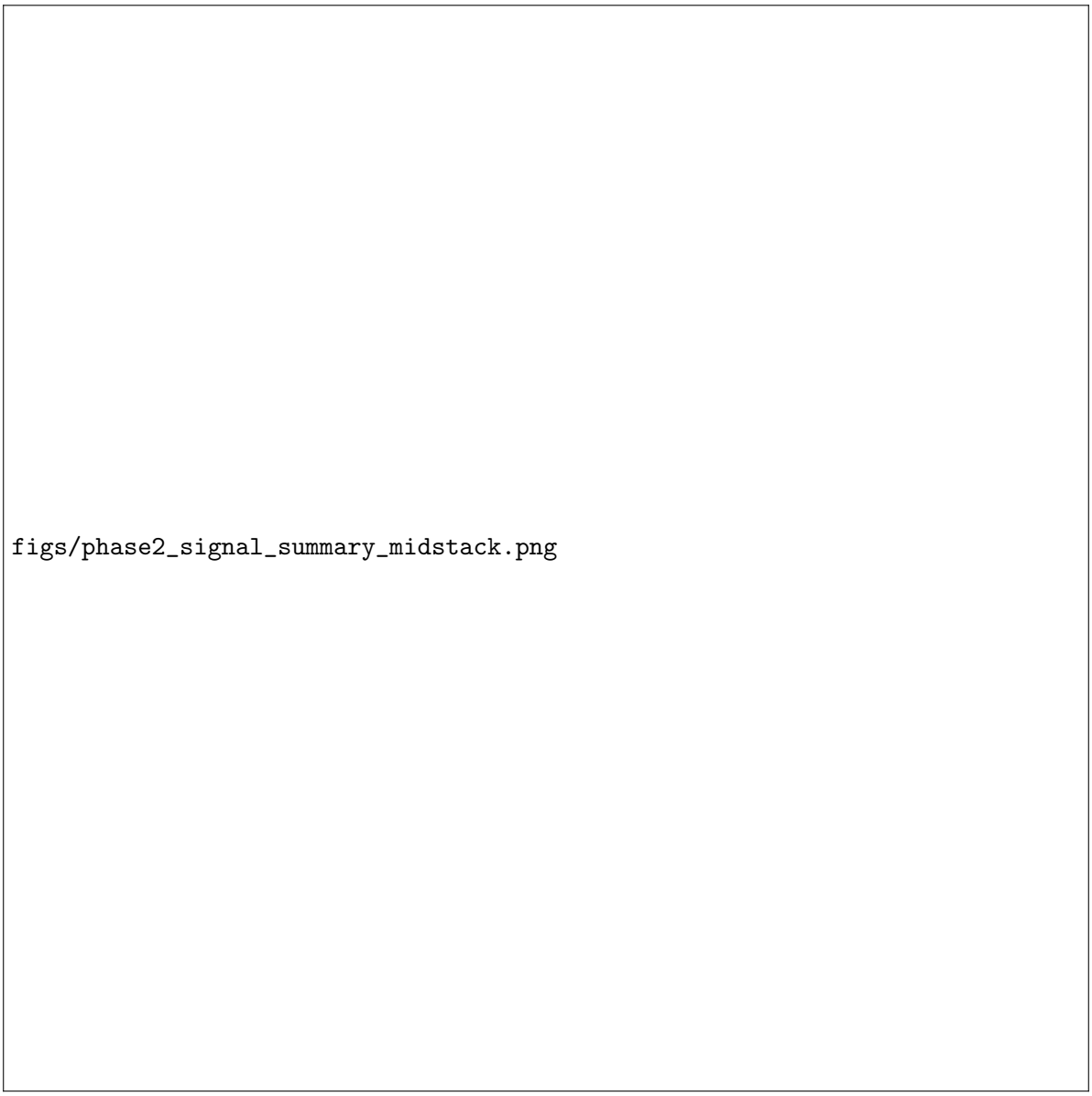


Figure 7: **Phase 2 mid-stack signal summary.** Same layout as Figure **??**. Contrast: drift magnitudes have converged across architectures (Phi 0.033, Gemma 0.106, Qwen 0.235 — within one order of magnitude rather than 20 as at last-layer), while variance now bifurcates with Qwen's L2-norm variance collapsing to 0.09 (essentially constant L2; the recurrent DeltaNet state's spectral structure is preserved through the mid-stack). The bar plot makes explicit that the trigger-feature choice should be *per-architecture*: drift carries the signal universally, variance is informative on Phi and Gemma but uninformative on Qwen at mid-stack.

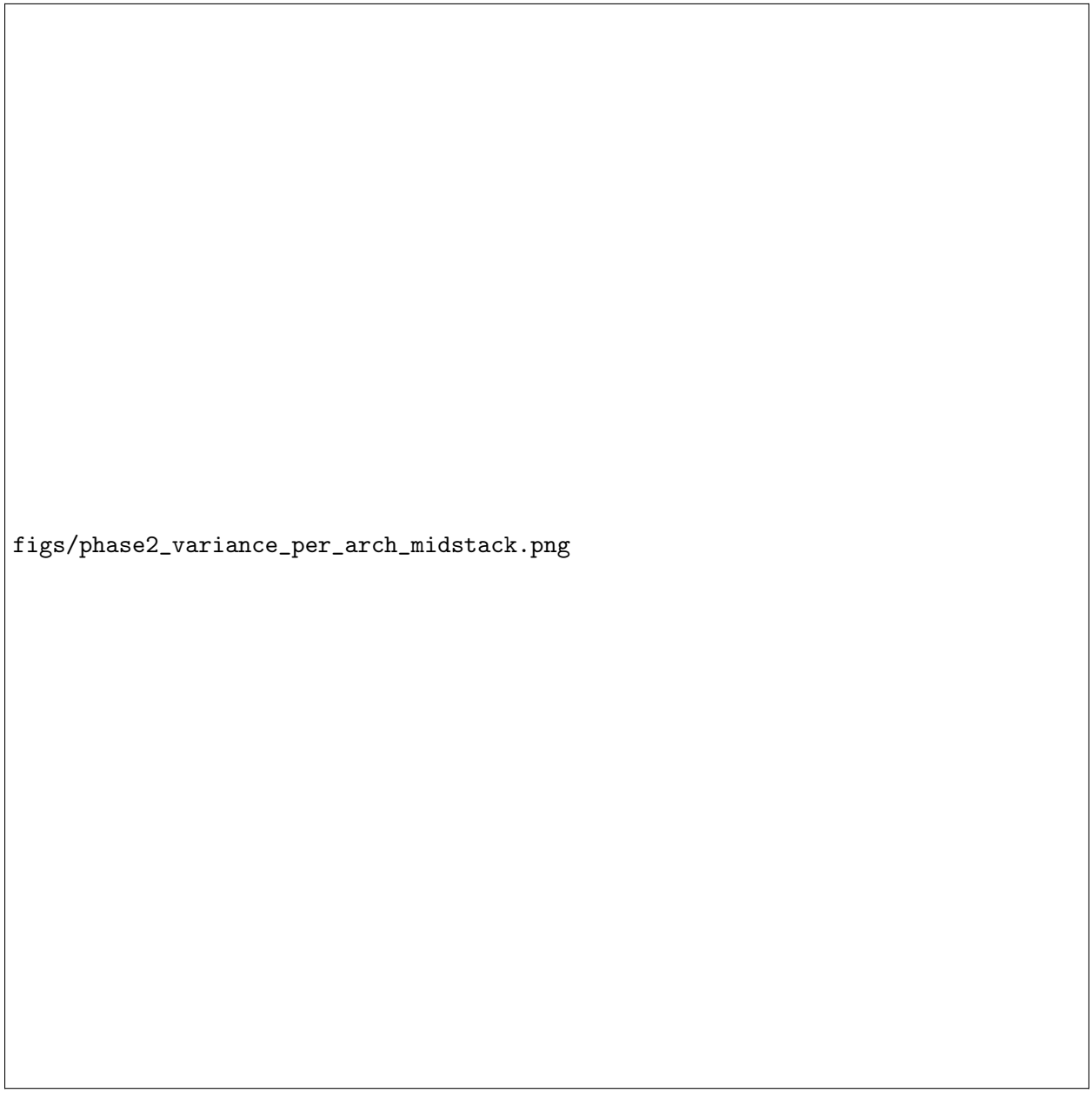


Figure 8: **Rolling L2-norm variance over** 5 000 **tokens, mid-stack mitigation.** Same row-and-line-style layout as Figure **??**. **Phi** (top): the strong event spike at $T \approx 4{,}000$ in the factual / multi-hop traces is preserved with sharper peak (max ∼120 vs ∼45 at last-layer); the rest of the trace is a flat baseline. **Gemma** (middle): a slowly rising multi-hop variance envelope is visible (dashed orange), and the factual / ambiguous traces show step-like structure rather than the noisy continuum at last-layer (the trace absolute scale is also much smaller, $\sim$ 0–700 vs $\sim$ 0–1500). **Qwen** (bottom): the variance of L2 norm has collapsed to the 0–0.35 range, four orders of magnitude smaller than last-layer. This confirms that *the per-token L2-norm of the mid-stack representation is essentially constant on Qwen3.6*; whatever signal exists at mid-stack lives in directional change ($\text{drift}_{\cos}$), not in norm fluctuation.